

Agent-Based Coding OS (Self-Driving Codebase AI)

Aftab Alam

Overview

This project aims to build an **autonomous AI software engineer** that can understand, modify, test, and improve entire codebases with minimal human intervention.

Instead of assisting developers with suggestions, the system executes tasks end-to-end, transforming high-level goals into working code.

Tagline:

"Give a goal. Get working code."

Problem Statement

Modern software development still faces significant challenges:

- Debugging consumes excessive time
- Project setup is repetitive and error-prone
- Large codebases are difficult to understand
- Small changes often break multiple components

Even with existing AI tools, developers still need to manually:

- Apply code changes
- Debug errors
- Verify execution

Solution

The system introduces an AI that completes tasks, not suggestions.

Core Idea:

Give a goal → AI executes → verifies → fixes → delivers working output

System Architecture

Execution Loop (Core Engine)

PLAN → EXECUTE → RUN → FIX → REPEAT → DONE
--

Key Components

1. Planner Agent

- Interprets user goals
- Breaks tasks into actionable steps

2. Builder Agent

- Generates and modifies code
- Works across multiple files

3. Debugger Agent

- Analyzes runtime errors
- Identifies root causes
- Applies fixes

4. Runner & Test System

- Executes the project
- Validates outputs
- Feeds failures back into the loop

5. Context Engine

- Maintains codebase understanding
- Enables relevant code retrieval

6. Tool Layer

- File operations (read/write/update)
- Shell execution (controlled)
- Dependency handling

Key Differentiators

Feature	Traditional Tools	This System
Code Suggestions	Yes	No
Autonomous Execution	No	Yes
Multi-file Changes	Limited	Full
Debugging Loop	No	Yes
Project Understanding	No	Yes
Execution Verification	No	Yes

Use Cases

- Fix broken Python/FastAPI projects
- Add authentication systems (e.g., JWT)
- Optimize database queries
- Refactor monolithic codebases

Technology Stack

Core:

- Python

AI Orchestration:

- LangGraph (execution loop)
- LangChain (tool integration)

Models:

- OpenAI / Gemini APIs
- Optional local models (Ollama)

System Tools:

- File system operations
- Shell execution
- AST parsing

UI (Planned):

- Desktop app (Electron + React)
- Monaco code editor

Current Focus

- Python + FastAPI projects
- Autonomous debugging loop
- CLI-based prototype

Roadmap

Phase 1:

- Autonomous Run & Fix system
- CLI interface

Phase 2:

- Multi-file feature execution
- Context-aware reasoning

Phase 3:

- Desktop UI
- Visual codebase mapping

Phase 4:

- Advanced agent collaboration
- Full automation workflows

Safety & Control

- Controlled tool access
- Execution safeguards
- Optional user confirmation for risky actions

Impact

- Reduces debugging time
- Simplifies project setup
- Accelerates development cycles
- Enables developers to focus on higher-level problem solving

Conclusion

This project represents a shift from AI-assisted development to autonomous problem-solving systems.

AI that does not just help developers, but actively solves development problems.